

Project Report Format

SMART LENDER – APPLICANT CREDIBILITY PREDICTION FOR LOAN APPROVAL

Project Description

The Smart Lender – Applicant Credibility Prediction for Loan Approval is an intelligent machine learning-based web application designed to help financial institutions evaluate loan applications efficiently and accurately. Traditional loan approval processes require manual verification of applicant details, making the process time-consuming and prone to human error. This project automates the evaluation process by analyzing applicant information using machine learning algorithms to predict whether a loan should be approved.

The system collects various applicant details such as gender, marital status, education, employment status, applicant income, co-applicant income, loan amount, loan amount term, credit history, and property area. These details are preprocessed and passed to a trained machine learning model that predicts the credibility of the applicant and recommends whether the loan application should be approved or rejected.

The project is developed using Python and the Flask framework for the backend, while HTML, CSS, and JavaScript are used for creating an interactive user interface. The trained machine learning model is integrated into the Flask application to provide real-time predictions. IBM Watsonx.ai is utilized for model development and experimentation, while GitHub is used for source code management and project documentation.

The Smart Lender system improves decision-making by minimizing manual effort, reducing approval time, and ensuring consistent evaluation of loan applications. The project demonstrates how Artificial Intelligence and Machine Learning can be effectively applied in the banking and financial sectors to enhance customer service and operational efficiency.

Application Scenarios

Scenario 1: Loan Approval in Banks

A customer visits a bank and submits a loan application. The bank employee enters the applicant's information into the Smart Lender application. The system analyzes the provided information using the trained machine learning model and predicts whether the applicant is eligible for loan approval. This reduces the time required for manual verification.

Scenario 2: Online Loan Application

A financial institution provides an online loan application portal. Customers fill in their details through the web application. The Smart Lender system instantly predicts loan eligibility and provides recommendations to the bank, making the online loan approval process faster and more efficient.

Scenario 3: Rural Banking Services

Banks serving rural customers can use Smart Lender to evaluate loan applications consistently. The system helps banking staff make data-driven decisions even in branches with limited technical resources.

Scenario 4: Financial Institutions and NBFCs

Non-Banking Financial Companies (NBFCs) can integrate the Smart Lender application into their existing systems to automate preliminary loan screening. This enables them to process a larger number of applications while maintaining accuracy and reducing operational costs.

Technical Architecture

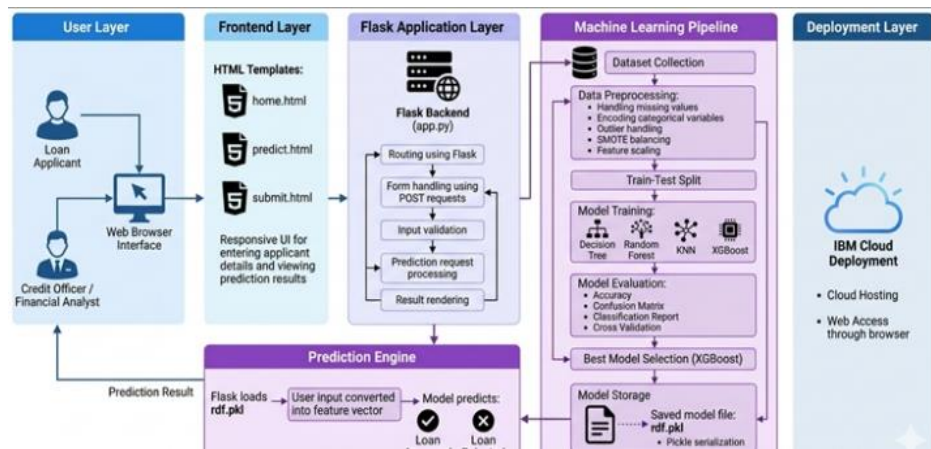
Description

The Smart Lender application follows a three-tier architecture consisting of the presentation layer, application layer, and machine learning layer.

The Presentation Layer is developed using HTML, CSS, and JavaScript, providing a user-friendly interface where applicants or bank officials can enter loan application details.

The Application Layer is built using the Flask framework in Python. It receives user input, validates the information, preprocesses the data, loads the trained machine learning model, and generates prediction results.

The Machine Learning Layer consists of the trained classification model developed using IBM Watsonx.ai and Python libraries. The model predicts loan approval based on applicant information. The prediction result is then returned to the Flask application and displayed to the user.



Prerequisites

Before developing the Smart Lender project, the following prerequisites should be available:

- Python 3.10 or above
- Flask Framework
- IBM Watsonx.ai Account
- Git and GitHub Account
- Visual Studio Code
- Machine Learning Libraries (NumPy, Pandas, Scikit-learn)
- HTML, CSS, and JavaScript

- Web Browser (Google Chrome or Microsoft Edge)

Prior Knowledge Required

To understand and develop the Smart Lender application, students should have knowledge of:

- Python Programming
- Machine Learning Fundamentals
- Data Preprocessing Techniques
- Classification Algorithms
- Flask Web Framework
- HTML, CSS, and JavaScript
- IBM Watsonx.ai Platform
- GitHub Version Control
- Basic Banking and Loan Approval Process

Project Objectives

The primary objectives of the Smart Lender project are:

- To automate the loan approval process using Machine Learning.
- To reduce manual effort in evaluating loan applications.
- To improve the speed and accuracy of loan approval decisions.
- To predict applicant credibility based on historical data.
- To provide a user-friendly web interface for loan prediction.
- To integrate machine learning models with Flask applications.
- To demonstrate the practical application of Artificial Intelligence in banking.
- To minimize financial risks by supporting data-driven decision making.

System Workflow

The Smart Lender application follows the workflow below:

1. User opens the Smart Lender web application.
2. User enters applicant details.
3. Flask application validates the entered information.

4. Data preprocessing is performed.
5. The trained machine learning model receives the processed data.
6. The model predicts loan approval status.
7. Flask receives the prediction result.
8. The result is displayed on the web interface.
9. The user views whether the loan is Approved or Rejected.

I am

a loan applicant

I'm trying to

get my loan approved quickly and easily

But

the approval process takes a long time

Because

manual verification and document checking are time-consuming

Which makes me feel

frustrated and uncertain

Problem Statement (PS)	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	a loan applicant	get my loan approved quickly and easily	the approval process takes a long time	manual verification and document checking are time-consuming	frustrated and uncertain
PS-2	a bank loan approver	evaluate loan applications accurately and faster	there are too many applications and risk of human error	data is in different formats and verification is done manually	overwhelmed and pressured to make correct decisions

MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

Introduction

The first milestone of the Smart Lender project focuses on understanding the problem statement, gathering project requirements, selecting the appropriate technologies, analyzing the dataset, and designing the overall system architecture. Proper requirement analysis ensures that the project objectives are clearly defined before implementation begins. During this phase, the development team identifies the functional and non-functional requirements, studies the loan approval process, and plans the workflow of the machine learning model and web application.

This milestone establishes the foundation for the entire project by defining how data flows through the application, selecting suitable machine learning algorithms, and preparing the environment for future development.

Activity 1.1: Requirement Analysis

The first activity involves understanding the existing loan approval process followed by banks and financial institutions. The project requirements are carefully analyzed to identify the information needed for predicting loan approval.

The following applicant details are considered:

- Gender
- Marital Status
- Dependents
- Education
- Self Employment Status
- Applicant Income
- Co-applicant Income
- Loan Amount
- Loan Amount Term
- Credit History
- Property Area

The collected requirements help in determining the features that significantly influence loan approval

decisions.

Activity 1.2: Dataset Collection

The Smart Lender project uses a loan prediction dataset containing historical information about loan applicants and approval decisions. The dataset includes demographic information, financial details, and previous loan approval outcomes.

The collected dataset serves as the foundation for training the machine learning model. Each record represents an applicant, while the target variable indicates whether the loan was approved.

The dataset is examined to understand:

- Number of records
- Number of attributes
- Data types
- Missing values
- Target variable
- Feature distribution

Proper dataset analysis improves model performance and prediction accuracy.

Name	Last commit message	Last commit date
..		
loan_prediction(1).csv	Add files via upload	4 days ago
loan_prediction.csv	Add files via upload	4 days ago
loan_prediction.xlsx	Add files via upload	4 days ago

Activity 1.3: System Architecture Design

After understanding the project requirements, the overall architecture of the Smart Lender application is designed.

The architecture consists of three major components:

Presentation Layer

This layer provides the graphical user interface developed using HTML, CSS, and JavaScript. Users enter applicant details through web forms.

Application Layer

The Flask backend receives user requests, validates input values, performs preprocessing, loads the trained machine learning model, and returns prediction results.

Machine Learning Layer

This layer contains the trained classification model developed using IBM Watsonx.ai and Python libraries. The model processes applicant information and predicts loan approval.

The interaction among these layers ensures smooth communication between the user interface and the prediction engine.

Activity 1.4: Technology Selection

Selecting appropriate technologies is essential for developing an efficient and scalable application.

The technologies selected for the Smart Lender project include:

Technology	Purpose
Python	Backend Programming
Flask	Web Framework
HTML	User Interface
CSS	Styling
JavaScript	Client-side Interaction
IBM Watsonx.ai	Model Development
Pandas	Data Processing

Technology	Purpose
NumPy	Numerical Computation
Scikit-learn	Machine Learning
Pickle	Model Serialization
GitHub	Version Control

Activity 1.5: Project Planning

The development activities are divided into multiple milestones to ensure systematic implementation. The planning phase includes:

- Understanding the project objectives
- Selecting appropriate tools
- Preparing the dataset
- Designing system architecture
- Developing the machine learning model
- Integrating the model with Flask
- Designing the user interface
- Testing the application
- Deploying the project
- Preparing documentation

A structured development plan helps complete the project efficiently while maintaining quality.

Milestone 1 Summary

During this milestone, the project requirements were analyzed, the dataset was collected and studied, the system architecture was designed, suitable technologies were selected, and the overall development plan was prepared. These activities established a strong foundation for building the Smart Lender application and ensured that all future development stages followed a well-defined workflow.

MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

Introduction

The second milestone focuses on setting up the development environment required for building the Smart Lender application. A proper development environment ensures smooth implementation, testing, and deployment of the project. During this phase, all necessary software, libraries, frameworks, and tools are installed and configured. The project structure is also created, allowing developers to organize files efficiently and manage the source code effectively.

This milestone also includes configuring IBM Watsonx.ai, installing Python packages, preparing the dataset, and establishing the project folder structure. These activities ensure that the machine learning model and Flask web application can be developed without any technical issues.

Activity 2.1: Installing Required Software

The first step is installing all software required for the project.

The following software is installed:

- Python 3.10 or later
- Visual Studio Code
- Git
- GitHub Desktop (Optional)
- IBM Watsonx.ai Access
- Google Chrome Browser

Python serves as the programming language for developing the machine learning model and Flask application. Visual Studio Code provides an integrated development environment (IDE), while Git is used for version control. IBM Watsonx.ai supports AI model development and experimentation.

Activity 2.2: Installing Python Libraries

After installing Python, the required libraries are installed using the pip package manager.

The important libraries include:

- Flask

- NumPy
- Pandas
- Scikit-learn
- Matplotlib
- Seaborn
- Pickle
- Joblib

These libraries support data preprocessing, machine learning model development, visualization, and deployment of the web application.

Example installation command:

```
pip install flask
pip install numpy
pip install pandas
pip install scikit-learn
pip install matplotlib
pip install seaborn
```

After successful installation, the libraries are imported into the project for further development.

Activity 2.3: Creating the Project Structure

A well-organized project structure improves readability, maintainability, and scalability.

The Smart Lender project contains the following folders:

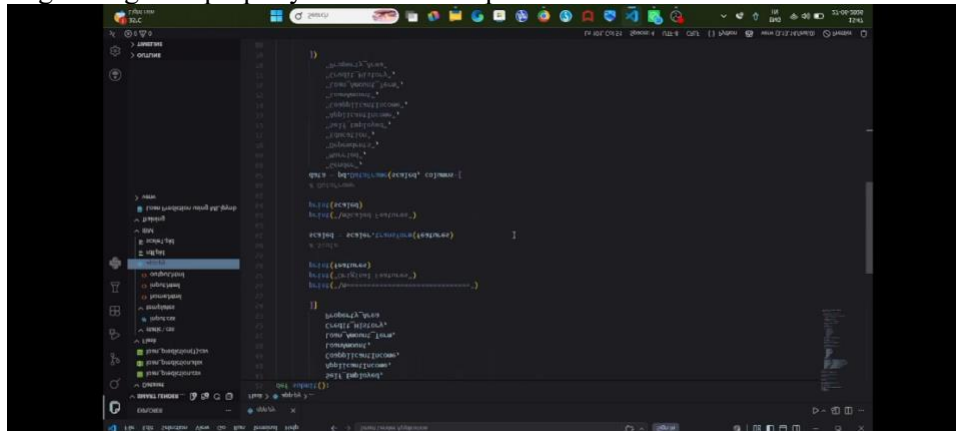
Smart-Lender/

```
├── dataset/
├── models/
├── static/
├── templates/
├── app.py
├── requirements.txt
├── README.md
└── trained_model.pkl
```

Each folder has a specific purpose.

- **templates** contains HTML pages.
- **static** stores CSS, images, and JavaScript files.
- **dataset** contains the loan prediction dataset.
- **model** stores trained machine learning models.
- **app.py** is the main Flask application.

Organizing files properly makes future updates easier.



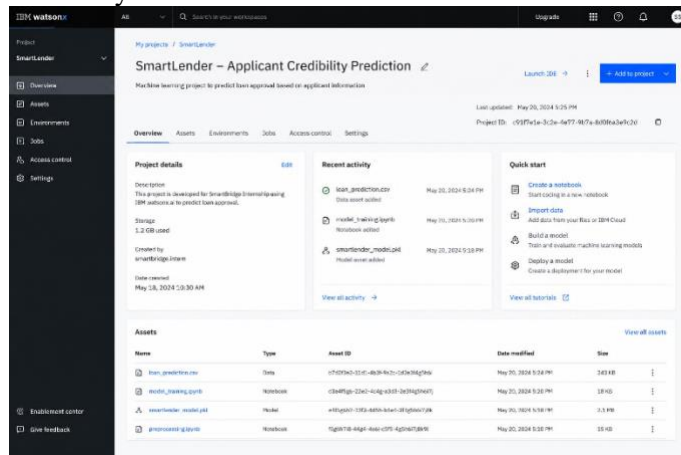
Activity 2.4: IBM Watsonx.ai Configuration

IBM Watsonx.ai is configured for machine learning model development.

The following steps are performed:

- Login to IBM Cloud.
- Open IBM Watsonx.ai Studio.
- Create a new project.
- Configure the project environment.
- Upload the loan prediction dataset.
- Verify successful dataset upload.
- Prepare the workspace for model development.

IBM Watsonx.ai provides cloud-based resources for building and testing machine learning models efficiently.



Activity 2.5: Dataset Preparation

The loan prediction dataset is uploaded into the development environment.

Before training the model, the dataset is carefully inspected.

The following preprocessing activities are performed:

- Checking missing values
- Handling null values
- Removing duplicate records
- Encoding categorical variables
- Normalizing numerical values
- Separating input features and target variable

Data preprocessing improves model accuracy by ensuring that the dataset is clean and suitable for training.

Activity 2.6: Environment Verification

After completing the setup, the environment is verified to ensure that all components are functioning correctly.

The verification process includes:

- Running Python successfully.
- Importing required libraries.
- Checking Flask installation.
- Opening the project in Visual Studio Code.
- Loading the dataset successfully.
- Verifying IBM Watsonx.ai workspace.
- Confirming GitHub repository connectivity.

Successful verification ensures that the development environment is ready for machine learning model implementation.

Milestone 2 Summary

In this milestone, the complete development environment required for the Smart Lender application was successfully established. Python, Flask, machine learning libraries, IBM Watsonx.ai, and Visual Studio Code were installed and configured. The project folder structure was created, the loan prediction dataset was uploaded and preprocessed, and the entire environment was verified. This

milestone provided a stable and organized platform for implementing and training the machine learning model in the next stage.

MILESTONE 3: MODEL DEVELOPMENT & TRAINING

Introduction

Milestone 3 focuses on developing the machine learning model that forms the core of the Smart Lender application. In this phase, the prepared dataset is analyzed, cleaned, and transformed into a suitable format for training. Different machine learning algorithms are evaluated to identify the model that provides the highest prediction accuracy. The selected model is then trained using historical loan application data and tested to ensure reliable predictions. Finally, the trained model is saved so that it can be integrated into the Flask web application.

Activity 3.1: Data Preprocessing

Before training the model, the dataset must be cleaned and prepared. Data preprocessing is an essential step because machine learning algorithms perform better when trained on accurate and consistent data.

The preprocessing activities include:

- Removing duplicate records.
- Handling missing values.
- Encoding categorical variables into numerical values.
- Normalizing numerical features.
- Verifying data consistency.
- Splitting features and target variables.

These steps improve the quality of the dataset and increase the accuracy of the prediction model.

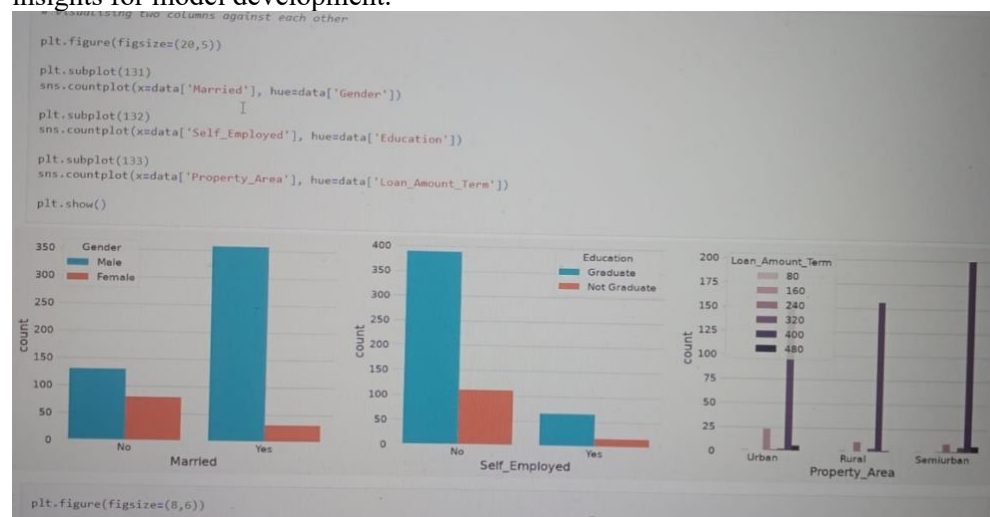
Activity 3.2: Exploratory Data Analysis (EDA)

Exploratory Data Analysis is performed to understand the characteristics of the dataset. Different visualizations and statistical methods are used to identify trends, relationships, and distributions among the variables.

The following analyses are carried out:

- Distribution of applicant income.
- Loan amount distribution.
- Credit history analysis.
- Property area comparison.
- Correlation among features.
- Loan approval distribution.

EDA helps identify important features that influence loan approval decisions and provides valuable insights for model development.



Activity 3.3: Feature Selection

Feature selection is the process of identifying the most relevant attributes that contribute to accurate loan approval prediction.

The selected features include:

- Gender
- Married
- Dependents
- Education
- Self Employed
- Applicant Income
- Co-applicant Income
- Loan Amount
- Loan Amount Term
- Credit History
- Property Area

The target variable is:

Loan Status

Selecting appropriate features improves model performance while reducing computational complexity.

Activity 3.4: Model Selection and Training

Several machine learning classification algorithms are evaluated to determine the most suitable model for loan approval prediction.

The algorithms considered include:

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier
- Support Vector Machine (SVM)
- Naïve Bayes

Each algorithm is trained using the prepared dataset and evaluated using appropriate performance metrics. After comparing their results, the model with the highest prediction accuracy and reliability is selected.

The selected model is then trained using the complete training dataset.

```
In [17]: # Importing and loading the Random Forest model
def RandomForest(X_train, X_test, y_train, y_test):
    model = RandomForestClassifier(X_train, y_train)
    y_train_pred = model.predict(X_train)
    print("Train Accuracy: ", accuracy_score(y_train, y_train_pred))
    y_test_pred = model.predict(X_test)
    print("Test Accuracy: ", accuracy_score(y_test, y_test_pred))
    return model

In [18]: rf_model = RandomForestClassifier(X_train, y_train, X_test, y_test)

Train Accuracy: 0.8
Test Accuracy: 0.7972222222222222

In [19]: # Importing and loading the SVM model
def SVM(X_train, X_test, y_train, y_test):
    model = SVC(kernel='rbf')
    model.fit(X_train, y_train)
    y_train_pred = model.predict(X_train)
    print("Train Accuracy: ", accuracy_score(y_train, y_train_pred))
    y_test_pred = model.predict(X_test)
    print("Test Accuracy: ", accuracy_score(y_test, y_test_pred))
    return model

In [20]: svm_model = SVM(X_train, X_test, y_train, y_test)

Train Accuracy: 0.8427777777777778
Test Accuracy: 0.7972222222222222

Out[20]: 0.7972222222222222
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with a browser.

In [21]: # Importing and loading the Gradient Boosting model
def GB(X_train, X_test, y_train, y_test):
    model = GradientBoostingClassifier(X_train, y_train)
    model.fit(X_train, y_train)
    y_train_pred = model.predict(X_train)
    print("Train Accuracy: ", accuracy_score(y_train, y_train_pred))
    y_test_pred = model.predict(X_test)
    print("Test Accuracy: ", accuracy_score(y_test, y_test_pred))
    return model

In [22]: gb_model = GB(X_train, X_test, y_train, y_test)

Train Accuracy: 0.8555555555555556
Test Accuracy: 0.7972222222222222

Out[22]: 0.7972222222222222
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with a browser.

In [23]: import pickle
pickle.dump(model, open("../data/RF.pkl", "wb"))
pickle.dump(svm_model, open("../data/SVM.pkl", "wb"))
pickle.dump(gb_model, open("../data/GB.pkl", "wb"))
```

Activity 3.5: Model Evaluation

After training, the model is tested using unseen data to evaluate its performance.

The evaluation metrics include:

- Accuracy
- Precision

- Recall
- F1-Score
- Confusion Matrix

These metrics help determine how effectively the model predicts loan approval outcomes. A higher accuracy indicates that the model performs well in classifying eligible and non-eligible applicants. Model evaluation ensures that the developed system is reliable before deployment.

Activity 3.6: Saving the Trained Model

Once the model achieves satisfactory performance, it is saved for future use in the Flask web application.

The trained model is serialized using Python's Pickle library. Saving the model eliminates the need for retraining every time the application is executed.

The saved model file is loaded by the Flask backend whenever a prediction request is received from the user.

This approach improves efficiency and reduces processing time during application execution.

Activity 3.7: Model Verification

The saved model is tested by providing sample applicant data to verify that it generates correct predictions.

The verification process includes:

- Loading the saved model.
- Providing sample applicant information.
- Running the prediction function.
- Comparing predicted results with expected outcomes.
- Confirming that the model works correctly.

Successful verification confirms that the trained model is ready for integration with the web application.

Milestone 3 Summary

In this milestone, the loan prediction dataset was preprocessed, analyzed, and prepared for machine learning. Relevant features influencing loan approval were selected, and multiple classification algorithms were evaluated. The best-performing model was trained, tested, and evaluated using standard performance metrics. After achieving satisfactory accuracy, the trained model was saved using the Pickle library and verified through sample predictions. This milestone successfully produced a reliable machine learning model that is ready for integration into the Smart Lender Flask application.

MILESTONE 4: APPLICATION DEVELOPMENT

Introduction

Milestone 4 focuses on developing the Smart Lender web application by integrating the trained machine learning model with a user-friendly web interface. The application is developed using the Flask framework, which acts as the backend server, while HTML, CSS, and JavaScript are used to create an interactive frontend. During this milestone, the application is designed to collect applicant information, process the input, send it to the trained machine learning model, and display the prediction results. This milestone transforms the trained model into a complete web application that can be used by banks and financial institutions for loan approval prediction.

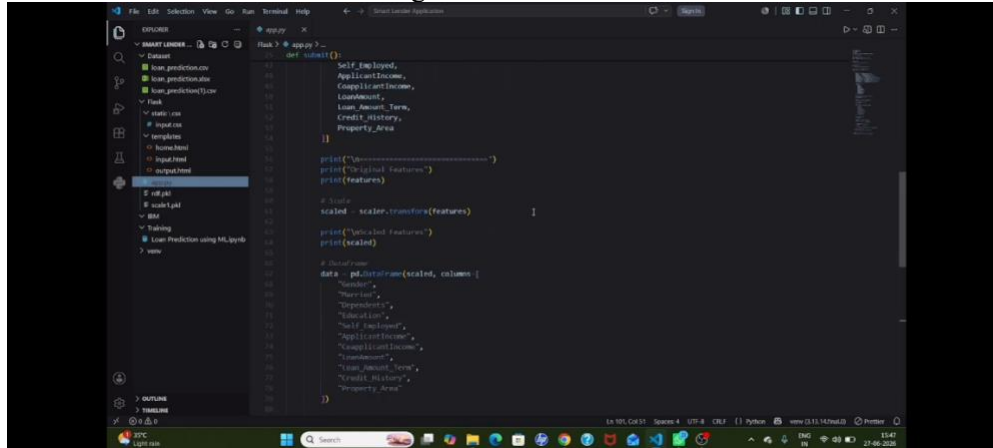
Activity 4.1: Developing the Flask Backend

The Flask framework is used to develop the backend of the Smart Lender application. The backend handles user requests, validates input data, loads the trained machine learning model, performs predictions, and sends the results back to the frontend.

The main responsibilities of the Flask backend include:

- Creating the Flask application.
- Defining application routes.
- Loading the trained machine learning model.
- Receiving user input from the web form.
- Validating input values.
- Passing the input to the prediction model.
- Returning the prediction result to the user interface.

The Flask backend acts as the bridge between the user interface and the machine learning model



Activity 4.2: Designing the User Interface

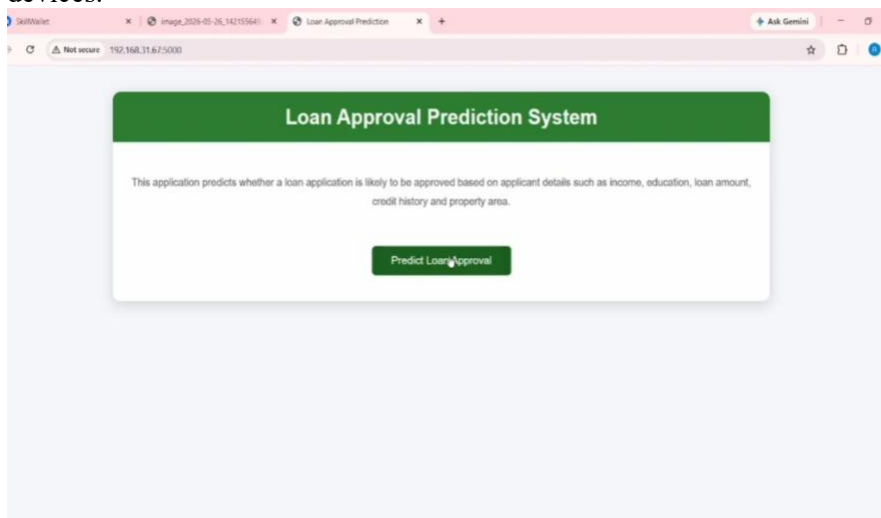
The frontend of the Smart Lender application is designed using HTML and CSS. The objective is to provide a clean, simple, and responsive interface that allows users to enter applicant details easily.

The interface contains input fields for:

- Gender
- Marital Status
- Dependents
- Education
- Self Employment
- Applicant Income
- Co-applicant Income
- Loan Amount
- Loan Amount Term
- Credit History
- Property Area

A "Predict Loan Status" button is provided to submit the entered information.

The responsive design ensures that the application works properly on desktop, tablet, and mobile devices.



Activity 4.3: Integrating the Machine Learning Model

After designing the frontend and backend, the trained machine learning model is integrated into the Flask application.

The integration process includes:

- Loading the saved model using Pickle.

- Reading user input from the HTML form.
- Converting input values into the required numerical format.
- Sending processed data to the prediction model.
- Receiving the prediction output.
- Displaying the prediction result.

The integration ensures smooth communication between the application and the trained machine learning model.

Activity 4.4: Input Validation

Input validation is implemented to ensure that users provide valid information before making predictions.

The application verifies:

- Mandatory fields are completed.
- Numerical values contain only valid numbers.
- Loan amount is within an acceptable range.
- Applicant income is valid.
- Credit history values are correctly selected.
- Invalid or empty inputs are rejected with appropriate messages.

Validation improves application reliability and prevents prediction errors.

Activity 4.5: Prediction Result Generation

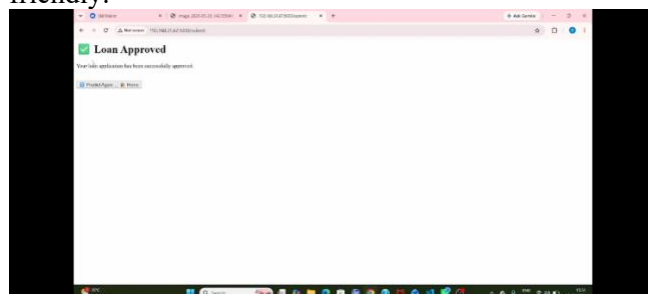
Once the user submits the application, the backend processes the information and sends it to the trained model.

The prediction model evaluates the applicant's credibility based on historical loan approval patterns and returns one of the following outcomes:

- Loan Approved
- Loan Rejected

The result is displayed clearly on the web page along with an appropriate message.

The prediction process is completed within a few seconds, making the application efficient and user-friendly.



Activity 4.6: Application Testing

The completed application is tested using multiple sample loan applications to verify that it functions correctly.

Testing activities include:

- Checking form submission.
- Verifying backend connectivity.
- Testing machine learning predictions.
- Validating different input combinations.
- Ensuring correct display of prediction results.
- Identifying and fixing minor errors.

Testing confirms that the application performs accurately under different scenarios.

Activity 4.7: User Experience Improvement

After testing, the application interface is improved to enhance user experience.

The improvements include:

- Better alignment of input fields.
- Improved button styling.
- Responsive layout.
- Clear prediction messages.
- Easy navigation.
- Consistent color scheme.
- Faster page loading.

These improvements make the application more attractive, efficient, and easier to use.

Milestone 4 Summary

In this milestone, the Smart Lender web application was successfully developed using the Flask framework, HTML, CSS, and JavaScript. The trained machine learning model was integrated into the application, enabling real-time loan approval prediction. User input validation, prediction result generation, responsive interface design, and application testing were completed successfully. The final application provides an efficient, accurate, and user-friendly platform for predicting loan approval based on applicant information.

MILESTONE 5: TESTING & DEPLOYMENT

Introduction

The final milestone focuses on testing the Smart Lender application and preparing it for deployment. Testing ensures that all modules of the application function correctly and produce accurate prediction results. After successful testing, the application is deployed and the source code is maintained using GitHub. This milestone verifies the overall reliability, usability, and performance of the Smart Lender system before its final demonstration and submission.

Activity 5.1: Functional Testing

Functional testing is performed to ensure that every feature of the application works as expected. Each module is tested individually and then integrated to verify complete system functionality.

The following functionalities are tested:

- User input form
- Input validation
- Machine learning prediction
- Display of prediction results
- Error handling
- Navigation between pages
- Overall application performance

The testing confirms that the application behaves correctly under different input conditions.

Activity 5.2: Performance Testing

Performance testing evaluates the speed and responsiveness of the application.

The following aspects are verified:

- Application loading time
- Prediction response time
- Backend processing speed
- User interface responsiveness
- Stability during multiple prediction requests

The Smart Lender application generates prediction results within a few seconds, ensuring a smooth user experience.

Activity 5.3: GitHub Repository Management

The complete Smart Lender project is uploaded to GitHub for version control and project management.

The repository contains:

- Source code
- HTML templates
- CSS files
- JavaScript files
- Machine learning model
- Dataset
- Documentation
- README file
- Demonstration video link

GitHub enables easy collaboration, project tracking, and secure storage of the project files.

My GitHub Repository Link

[aasrithadarsi2006-star/smart-lender](https://github.com/aasrithadarsi2006-star/smart-lender)

Activity 5.4: Project Demonstration

After completing development and testing, the Smart Lender application is demonstrated to verify its functionality.

The demonstration includes:

- Opening the application
- Entering applicant details
- Submitting the loan application
- Processing user input
- Displaying loan approval prediction
- Explaining project workflow
- Demonstrating the GitHub repository

The demonstration confirms that all project objectives have been successfully achieved.

Activity 5.5: Final Deployment

The Smart Lender application is prepared for deployment and submission.

The deployment process includes:

- Final code verification
- Organizing project files
- Uploading the source code to GitHub
- Updating the README documentation
- Uploading the demonstration video
- Verifying all project links
- Preparing the final report

These activities ensure that the project is complete and ready for evaluation

Milestone 5 Summary

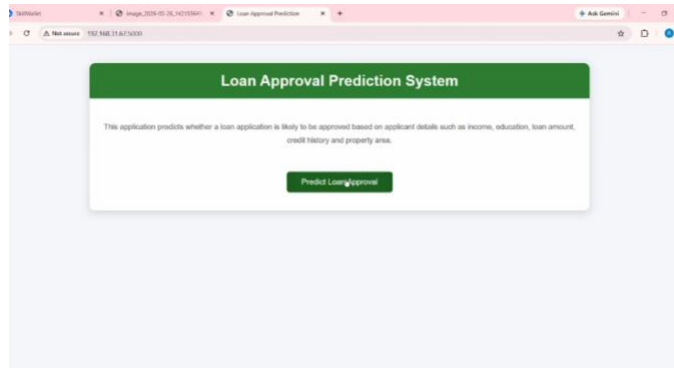
In this milestone, the Smart Lender application was thoroughly tested to ensure accurate predictions and reliable performance. Functional testing, performance testing, project demonstration, and deployment activities were completed successfully. The source code and project documentation were uploaded to GitHub, making the project ready for final submission and evaluation.

EXPLORING THE WEBSITE / APPLICATION PAGES

Home Page

Description

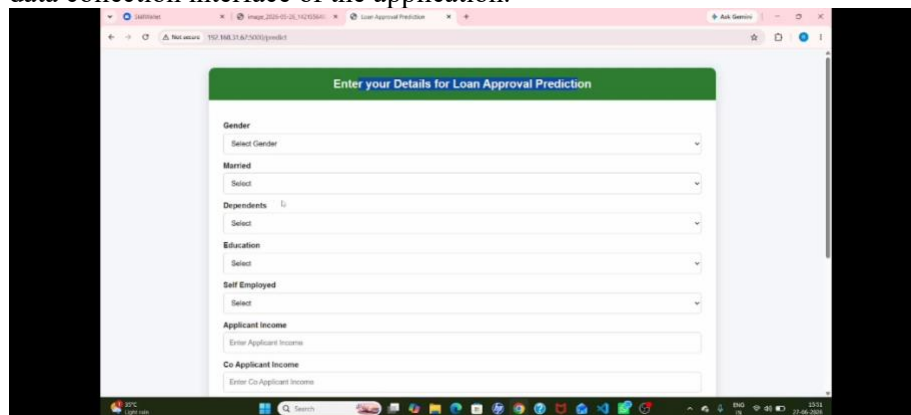
The Home Page serves as the main interface of the Smart Lender application. It welcomes users and provides a simple, user-friendly environment for entering applicant information. The page contains input fields, labels, and buttons that allow users to begin the loan prediction process. The responsive layout ensures accessibility across different devices.



Loan Application Page

Description

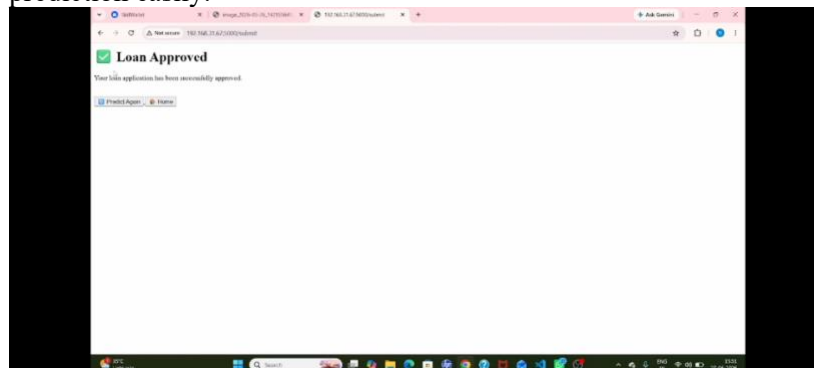
The Loan Application Page allows users to enter all required applicant details such as gender, marital status, education, income, loan amount, loan term, credit history, and property area. The entered information is validated before being submitted to the prediction model. This page acts as the primary data collection interface of the application.



Prediction Result Page

Description

The Prediction Result Page displays the outcome generated by the machine learning model. Based on the applicant's details, the system predicts whether the loan application is likely to be approved or rejected. The result is presented clearly with an informative message, allowing users to understand the prediction easily.



CONCLUSION

The **Smart Lender – Applicant Credibility Prediction for Loan Approval** project successfully demonstrates the practical application of Machine Learning and Artificial Intelligence in the banking and financial sector. The developed system automates the loan approval process by analyzing applicant information and predicting loan eligibility with improved speed and consistency. By integrating a trained machine learning model with a Flask-based web application, the project provides an efficient and user-friendly solution for evaluating loan applications.

Throughout the project, various stages such as requirement analysis, environment setup, data preprocessing, model development, application integration, testing, and deployment were successfully completed. The use of Python, Flask, HTML, CSS, JavaScript, IBM Watsonx.ai, and Scikit-learn enabled the development of a reliable application capable of making real-time loan approval predictions.

The Smart Lender application minimizes manual effort, reduces processing time, and supports better decision-making for financial institutions. The project highlights the importance of data-driven solutions in modern banking and demonstrates how machine learning can improve operational efficiency and customer service.

In the future, the application can be enhanced by integrating advanced machine learning algorithms, real-time banking databases, user authentication, cloud deployment, and explainable AI techniques to provide more transparent and accurate loan approval decisions. These improvements will further increase the system's reliability, scalability, and practical usability in real-world financial environments.